Volume xx (200y), Number z, pp. 1–4

Real-Time Image-Based Lighting with the Split-Sum Approximation

Vid Potočnik[1]

[1]University of Ljubljana, Faculty of Computer and Information Science, Slovenia

Abstract

We describe a real-time renderer that lights 3D objects using a high-dynamic-range (HDR) photograph of a real environment as the only light source. Doing this accurately would require an expensive numerical integration for every pixel of every frame, which is too slow for interactive use. Instead, the renderer precomputes three small textures from the environment image (a diffuse irradiance map, a roughness-indexed specular map, and a 2D lookup table) and combines them at render time with three texture fetches per pixel. The implementation runs at thousands of frames per second on an Apple Silicon laptop (the per-pixel IBL math is below the measurement floor) and lets the user switch HDR environments in 100–160 ms.

CCS Concepts

•***Computing methodologies*** → ***Rendering;*** *Reflectance modeling;*

1. Introduction

The appearance of a physical object depends as much on its surroundings as on the object itself: a chrome ball looks completely different in a sunlit street than in a cloudy room. To make a virtual object look real, the renderer has to account for the actual light coming from every direction. Image-based lighting (IBL), introduced by Debevec [Deb98, Deb06], captures that "every direction" once with an HDR panoramic photograph and uses it as the sole light source in the scene. The result is a virtual object that looks like it belongs in a photograph of the captured environment.

The problem is speed. The correct way to use the panorama is to integrate contributions from many directions per pixel, which is essentially a mini-raytracer at every shading sample, and is far too slow for interactive applications. Karis [Kar13] proposed a now-standard workaround: the expensive integral can be split into two factors that can each be precomputed separately and recombined cheaply at render time. This is the split-sum approximation, and it is the foundation of the real-time physically based rendering (PBR) pipelines used in modern game engines [Bur12].

This report describes such a pipeline. Section 2 explains what each piece is and why the split-sum works. Section 3 walks through the four precomputation passes and the runtime shader. Section 4 covers the implementation choices. Section 5 reports timings on the actual hardware and discusses the practical trade-offs.

2. Background

The job of a renderer is to compute the colour that leaves a surface point and travels toward the camera. For physical correctness this colour is the sum of all the light arriving at that point from every direction, multiplied by how much of each direction the material reflects toward the camera. Written as an integral, this is the rendering equation:

$$L_o(\omega_o) = \int_\Omega f_r(\omega_i, \omega_o)\, L_i(\omega_i)\, (\mathbf{n} \cdot \omega_i)\, \mathrm{d}\omega_i. \tag{1}$$

L_i is the incoming light from direction ω_i, L_o is the outgoing light, f_r is the material's reflectance, and $\mathbf{n}$ is the surface normal. Under IBL, L_i is read straight from the HDR panorama.

We use the Cook–Torrance microfacet model [CT82] for f_r, which is the industry standard for physically based shading. It splits the material's response into a matte (diffuse) lobe and a shiny (specular) lobe. The specular lobe is parameterised by a roughness value α that controls how blurred reflections look, and by the Fresnel coefficient that controls how much light reflects at grazing angles. The diffuse lobe is suppressed for metallic materials. The specific functions used are GGX for the microfacet distribution [WMLT07], Smith for the geometry term [Hei14], and Schlick's approximation for Fresnel.

The reason equation (1) is hard is that L_i is an arbitrary HDR image with no closed-form expression, so the integral has no analytic solution and must be estimated numerically. Doing this fairly takes many samples per pixel, which is incompatible with real-time use.

The matte lobe has a simple workaround: it averages incoming light over the whole hemisphere weighted by a smooth cosine kernel, so the result depends only on the surface normal. We can precompute this average for every direction once and store it as a cubemap, the *irradiance map*. At render time the diffuse contribution is then one texture fetch.

The specular lobe is harder because the result depends on both

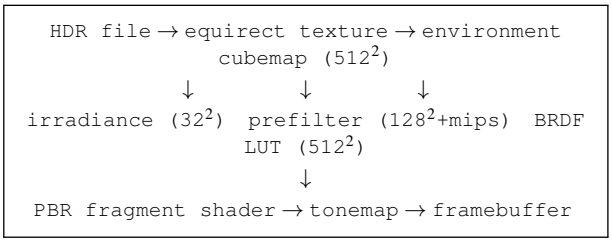

Figure 1: *Pipeline overview. Four passes run once when an HDR environment is loaded. The rendering pass runs once per frame and reads the three precomputed maps plus the BRDF LUT.*

the surface normal *and* the camera direction (so a moving camera sees a moving reflection). Karis observed that this integral can be *approximately* factored into two independent integrals: one that depends on the lighting and roughness but not on the material, and one that depends on the material and roughness but not on the lighting. Schematically,

$$\text{specular IBL} \approx \underbrace{\text{(prefiltered light)}}_{\text{(I)}} \cdot \underbrace{\text{(BRDF integral)}}_{\text{(II)}}. \qquad (2)$$

Factor (I) is the environment blurred by a roughness-dependent kernel; for each roughness it is again precomputed into a cubemap, with different roughnesses stored as mip-levels of the same texture. Factor (II) reduces to two scalars that depend only on (viewing angle, roughness) and is precomputed into a small 2D lookup table, the *BRDF LUT*. Both factors together let us evaluate the specular IBL contribution with two more texture fetches at render time.

3. Pipeline

The pipeline (Fig. 1) has four precomputation passes that run once per environment, and a fragment shader that runs every frame.

3.1. Equirectangular–to–cubemap

HDR files store the panorama as a flat 2:1 image (latitude × longitude). Cubemaps are easier and faster to sample on the GPU, so we render the equirectangular image onto the six faces of a 512^2 cubemap. The fragment shader for this pass takes the world-space direction of each output texel and converts it to UV coordinates in the source image. The resulting cubemap is stored as 16-bit floats per channel so the HDR range survives.

3.2. Diffuse irradiance

For each output texel of a small (32^2) cubemap, we average the incoming light over the upper hemisphere around that direction with a cosine weight. The integration is a uniform Riemann sum on the two spherical angles, with a step of 0.025 rad (≈ 1300 samples per output texel). The low resolution is fine because the cosine kernel is a strong low-pass filter; any high-frequency detail would be averaged away anyway.

3.3. GGX specular prefilter

The prefiltered cubemap is allocated at 128^2 with five mip levels. Each mip corresponds to a fixed roughness: mip 0 is a perfect mirror ($\alpha = 0$), mip 4 is fully rough ($\alpha = 1$). For every output texel of every mip, the shader integrates the environment with a GGX-shaped kernel using importance sampling, drawing samples from a Hammersley quasi-random sequence [HH64] so that more samples land where the kernel has more weight. The default is 1024 samples per texel.

A small caveat: the formal specular integrand depends on both the surface normal and the viewing direction, but the prefilter step has to commit to one of them. Karis [Kar13] chooses $\mathbf{n} = \mathbf{v}$, which makes the precomputation tractable at the cost of slightly incorrect reflections at oblique viewing angles. In practice the error is masked by the high-frequency content of natural HDR images.

3.4. BRDF lookup table

The remaining material-dependent factor reduces to two scalar functions of $(\mathbf{n}\cdot\omega_o, \alpha)$ that have no closed form but can be evaluated by Monte Carlo. We render a single full-screen quad into a 512^2 `RG16F` texture; the fragment shader integrates the geometry/Fresnel response with 1024 GGX importance samples and writes the two factors into the R and G channels. This LUT is independent of the HDR environment and the material, so it is built once at startup and reused for every environment and every material.

3.5. Runtime shading

Per pixel the shader does the following: compute the view-reflection direction R, fetch the diffuse irradiance from the irradiance cubemap along the surface normal, fetch the prefiltered reflection from the prefilter cubemap along R at a mip level chosen from the roughness, and fetch the BRDF LUT at $(\mathbf{n}\cdot\mathbf{v}, \alpha)$. The three contributions are combined with a roughness-modulated Schlick Fresnel [Laz13] that avoids edge darkening at high roughness. The final colour is multiplied by an exposure factor, passed through a tonemapping operator (Reinhard, ACES, or Uncharted 2 [Hab10]), and gamma-corrected.

Listing 1: *Core of the per-pixel IBL evaluation.*

```
vec3 F = fresnelSchlickRoughness(NdotV, F0, roughness);
vec3 kD = (1.0 - F) * (1.0 - metallic);

vec3 irradiance = texture(irradianceMap, N).rgb;
vec3 diffuse = kD * irradiance * albedo;

vec3 prefiltered = textureLod(prefilterMap, R, roughness * 4.0).
     rgb;
vec2 brdf = texture(brdfLUT, vec2(NdotV, roughness)).rg;
vec3 specular = prefiltered * (F * brdf.x + brdf.y);

color = (diffuse + specular) * exposure;
color = tonemap(color);
```

4. Implementation

The renderer is written in C++17 against OpenGL 3.3 core profile. GLFW provides the window and OpenGL context, GLAD loads the

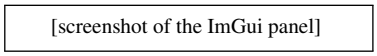

Figure 2: *ImGui control panel. The user adjusts material parameters, exposure, the tonemapping operator, scene composition, the active HDR environment, and the four IBL preprocessing resolutions.*

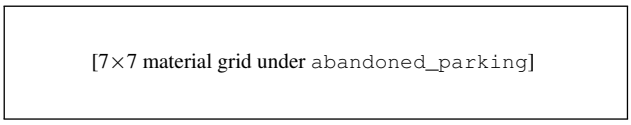

Figure 3: *Material grid. Rows top–to–bottom: increasing metallic. Columns left–to–right: increasing roughness. A single HDR environment (`abandoned_parking`) is used for all 49 spheres.*

GL functions, `stb_image` decodes the HDR files, GLM handles linear algebra, and Dear ImGui provides the control panel.

All four precomputation parameters (cubemap resolution, irradiance resolution, prefilter resolution, prefilter sample count) are exposed in the ImGui panel. Adjusting any of them triggers regeneration of just the affected map, so the user can interactively trade preprocessing cost for quality without restarting the application. The scene offers three primitive meshes (sphere, cube, torus) and a 7×7 material-grid mode that displays a full sweep of roughness and metallic values at once.

5. Results

5.1. What it looks like

The material grid in Fig. 3 shows the full sweep of roughness (left to right) and metallic (top to bottom) for one HDR environment. Dielectrics in the top row reflect the environment with a soft, roughness-dependent blur and exhibit Fresnel rim brightening; metals in the bottom row have a coloured specular response controlled by the albedo and no diffuse contribution. Intermediate rows interpolate smoothly between the two regimes, as expected from the metallic workflow.

Fig. 4 shows a single near-mirror sphere under three HDR environments with very different lighting character. The renderer reproduces the soft overcast of `abandoned_parking`, the directional studio lighting of `studio_small_03`, and the bright sunlit `kloppenheim_06`. Because the environment is stored in 16-bit float, the sun disk in the last environment retains its true intensity and produces a clipped highlight after tonemapping, just as a photographed reflection would.

5.2. Preprocessing cost

We measured wall-clock times around `glFinish()` barriers to capture actual GPU execution cost, on an Apple Silicon Mac running OpenGL 4.1 via the Metal driver (driver version 90.5). Table 1 sweeps the four parameters to isolate the cost of each pass.

The numbers confirm what the algorithms predict. Doubling the prefilter output resolution (rows 3→4) increases the prefilter cost by about 50 %, not by $4\times$, because only the largest mip level grows; the smaller mips dominate the texel count but stay the same. Doubling the prefilter sample count (rows 4→5) almost exactly doubles the prefilter cost. The irradiance pass scales with output resolution because each output texel does the same fixed-cost Riemann integration. The BRDF LUT is constant at around 8 ms regardless of any other parameter because it does not depend on the environment.

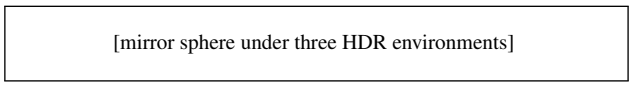

Figure 4: *Mirror sphere ($\alpha = 0.05$, $m = 1$, white albedo) under three HDR environments: `abandoned_parking`, `studio_small_03`, and `kloppenheim_06` (left to right).*

Even at the most expensive operating point the total preprocessing cost is under 200 ms, which is fast enough that switching the HDR environment from the UI feels instant.

5.3. Rendering cost

To measure actual shader cost we disable vsync and time individual frames with a `glFinish()` barrier and a CPU timestamp. Each configuration runs for 120 warmup frames followed by 3000 measured frames. Because the distribution has occasional long-tail outliers from OS scheduling and window-system activity, we report the *median* frame time as the headline number, with the 1st and 99th percentile as a robustness check. Table 2 compares four configurations: a single sphere or the 7×7 material grid, each with full IBL (diffuse + specular) or diffuse-only.

The single-sphere medians (0.217 vs 0.230 ms) and the grid medians (0.425 vs 0.423 ms) differ by less than the spread of either distribution, which means the per-pixel IBL evaluation (the three texture fetches plus the Cook–Torrance arithmetic) is too cheap to be measurable at this resolution: enabling or disabling specular IBL does not produce a detectable change in frame time. The frame cost is dominated by the fixed per-frame overhead (clear, swap, UI) in the single-sphere scene, and by geometry processing in the 49-sphere grid scene. In both cases the application sustains thousands of frames per second — between roughly 2300 FPS for the grid and 4500 FPS for the single sphere — confirming that equation (2) delivers real-time performance with significant headroom.

5.4. Trade-offs

The free parameters affect quality and cost asymmetrically. The cubemap resolution sets a ceiling on the angular detail of all the downstream maps, but going from 512^2 to 1024^2 has little visible effect. The irradiance resolution can be very low (16–64) because of the cosine kernel; doubling it is essentially free in image quality. The prefilter resolution and sample count matter the most: too few samples leave visible high-frequency noise at moderate roughness, and too low a resolution clips sharp reflections at low roughness. The BRDF LUT can be modest (256 to 512) because it is sampled with bilinear filtering and contains very smooth functions.

Of the four parameters, only the prefilter cost scales meaningfully: linearly with sample count and roughly quadratically with

Table 1: *Preprocessing time (ms) at five operating points on an Apple Silicon Mac. Row 1 includes a one-time shader-compilation overhead on first use; rows 2–5 are warm.*

#	Cubemap	Irradiance	Prefilter	Samples	Equirect	Irradiance	Prefilter	BRDF LUT	Total
1 (cold)	512^2	32^2	128^2	1024	59.8	175.9	43.3	21.2	300.1
2	1024^2	32^2	128^2	1024	29.4	43.5	22.0	8.1	102.9
3	1024^2	64^2	128^2	1024	26.2	65.4	21.8	7.9	121.4
4	1024^2	64^2	256^2	1024	57.9	63.2	33.5	7.9	162.5
5	1024^2	64^2	256^2	2048	23.7	67.2	63.2	7.9	154.0

Table 2: *Per-frame render time (ms) at* 1280×720 *with vsync off on the same Apple Silicon Mac. Statistics from 3000 measured frames per configuration.*

Configuration	median	p1	p99	FPS
single, full IBL	0.217	0.190	1.764	4604
single, diffuse only	0.230	0.192	1.963	4357
grid, full IBL	0.425	0.397	1.266	2352
grid, diffuse only	0.423	0.390	1.292	2365

the largest mip resolution. The others can be set generously without penalty.

6. Limitations and Future Work

The renderer models a single bounce of environmental light. It does not account for parallax inside the environment (the HDR is treated as infinitely far away), local shadowing of the IBL contribution (which would require an ambient-occlusion or screen-space visibility pass), or interreflection between scene objects.

The diffuse irradiance is integrated as a Riemann sum on a cubemap; the classical alternative is to project it onto spherical harmonics [RH01], which replaces a cubemap by nine RGB coefficients with no perceptible loss.

The Karis prefilter assumption $\mathbf{n} = \mathbf{v}$ is the main source of approximation error in the specular term. A more accurate but still real-time alternative is the multi-scattering correction of Fdez-Agüera [FA19], which is important for high-roughness metals where single-scattering Smith geometry visibly loses energy.

7. Conclusion

We described a real-time IBL pipeline based on the split-sum approximation, implemented it in OpenGL, and measured its behaviour on commodity laptop hardware. The renderer sustains thousands of frames per second at 1280×720 in both the single-sphere and 49-sphere grid scenes, with the per-pixel IBL evaluation itself being too cheap to measure, and switches HDR environments in well under 200 ms. The pipeline exposes four intuitive parameters that the user can adjust live; only one of them (the prefilter sample count) is expensive enough to be worth tuning carefully. The source code, build instructions, and the data used to produce this report are available under the MIT licence.

References

[Bur12] BURLEY B.: Physically-based shading at Disney. In *ACM SIGGRAPH 2012 Courses: Physically Based Shading in Theory and Practice* (2012). 1

[CT82] COOK R. L., TORRANCE K. E.: A reflectance model for computer graphics. *ACM Transactions on Graphics 1*, 1 (1982), 7–24. 1

[Deb98] DEBEVEC P. E.: Rendering synthetic objects into real scenes: Bridging traditional and image-based graphics with global illumination and high dynamic range photography. In *Proceedings of SIGGRAPH '98* (1998), pp. 189–198. 1

[Deb06] DEBEVEC P. E.: Image-based lighting. In *ACM SIGGRAPH 2006 Courses* (2006). 1

[FA19] FDEZ-AGÜERA C. J.: A multiple-scattering microfacet model for real-time image-based lighting. *Journal of Computer Graphics Techniques 8*, 1 (2019), 45–55. 4

[Hab10] HABLE J.: Uncharted 2: HDR lighting. In *Game Developers Conference* (2010). 2

[Hei14] HEITZ E.: Understanding the masking-shadowing function in microfacet-based BRDFs. *Journal of Computer Graphics Techniques 3*, 2 (2014), 48–107. 1

[HH64] HAMMERSLEY J. M., HANDSCOMB D. C.: *Monte Carlo Methods*. Methuen, London, 1964. 2

[Kar13] KARIS B.: Real shading in Unreal Engine 4. In *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice* (2013). 1, 2

[Laz13] LAZAROV D.: Getting more physical in Call of Duty: Black Ops II. In *ACM SIGGRAPH 2013 Courses: Physically Based Shading in Theory and Practice* (2013). 2

[RH01] RAMAMOORTHI R., HANRAHAN P.: An efficient representation for irradiance environment maps. In *Proceedings of SIGGRAPH '01* (2001), pp. 497–500. 4

[WMLT07] WALTER B., MARSCHNER S. R., LI H., TORRANCE K. E.: Microfacet models for refraction through rough surfaces. In *Proceedings of the Eurographics Symposium on Rendering* (2007), pp. 195–206. 1